Accurate Interpolation of Library Timing Parameters Through Recurrent Convolutional Neural Network

Daijoon Hyun, *Member, IEEE*, Younggwang Jung, *Graduate Student Member, IEEE*, and Youngsoo Shin, *Fellow, IEEE*

***Abstract*—Interpolation is used to approximate the timing parameters of logic cells not specified in timing tables. Bilinear interpolation has been taken for granted in the industry, but the error increases as the nonlinearity of the timing parameters increases. In this article, we propose machine learning (ML)-based interpolation to obtain more accurate timing parameters. Recurrent convolutional neural network (R-CNN) is employed and various ranges of table entries form a sequence of input data, in which the recurrent network allows them to influence the interpolation. In addition, variational autoencoder (VAE) is used to capture the distribution feature of the table. ML interpolation is parallelized in GPU to minimize the runtime overhead from numerous arithmetic operations. Experimental results demonstrate that ML interpolation reduces timing parameter error by 19.7% and path delay error by 3.4% compared to bilinear interpolation at the cost of 13% runtime overhead.**

***Index Terms*—Interpolation, recurrent convolutional neural network (R-CNN), timing analysis, timing library, variational autoencoder (VAE).**

I. Introduction

A digital integrated circuit is designed and analyzed through a technology library. Each cell or macro in a library is characterized for its timing, power, and so on, through table entries. Pin-to-pin delay, for example, is modeled by a 7-by-7 table indexed by load capacitance and input transition time. Missing table entry, upon request from timing analysis, is extracted through interpolation with four neighbor entries as [1]. A popular bilinear interpolation is not accurate enough, especially when load capacitance or input transition time is large. Our test indicates that cell delay can be inaccurate by 47 ps in case of INV in 55-nm technology; the error of path delay can be as high as 9%.

Recently, there have been a few approaches to improving the accuracy and runtime of timing analysis using machine learning (ML) models. One method uses multilayer perceptron (MLP) and autoencoder to predict the output waveform of logic cell from an input waveform [2], where the autoencoder encodes the input waveform into a few parameters that become MLP input and decodes the output of MLP to the output waveform. This yields errors of up to 19% in INV delay even though a significant number of simulations are used as training samples. In [3], MLPs are employed to refine initial timing estimation at the driving point, which greatly speeds up timing analysis but the accuracy may vary depending on whether the interconnect structure is included in training.

In this article, we propose ML-based interpolation, ML interpolation for short, to minimize the accuracy loss in interpolating table entries. This is the first method of applying ML model to the interpolation of library parameters. We introduce recurrent convolutional neural network (R-CNN) to predict unknown timing values, where the table entries considered for each layer gradually narrow from a wide range to only around input conditions, such as input transition time and load capacitance, and recurrent networks allow various ranges of table entries to affect the interpolation. In addition, variational autoencoder (VAE) is used to capture the distribution features of table entries, which are then employed as input features for R-CNN. Since R-CNN requires a lot of arithmetic operations, the runtime of static timing analysis (STA) increases significantly. In order to minimize the runtime overhead, most of the operations are conducted beforehand, and ML interpolations are parallelized in GPU.

The remainder of this article is organized as follows. Section II introduces input features and ML model used for ML interpolation. In Section III, the effectiveness of ML interpolation is assessed in terms of the accuracy of the predicted timing values, the effect on path delays, and STA runtime. Conclusions are drawn in Section IV.

II. ML Interpolation

The timing library includes the timing parameters of individual cells, such as cell delay, output transition time, and timing constraints, that describe the timing characteristic of standard cells. They are characterized by simulation and stored in a timing table indexed by the input conditions. Our problem is to predict unknown timing parameters for given input conditions using table entries, where the goal is to minimize errors compared to the exact simulation results.

A. Input Features

There are three types of input features: input conditions of the gate, three input maps, and distribution vectors encoded from a timing table. Fig. 1 illustrates three input maps, such as timing parameter map, row-index map, and column-index map. They have 8×8 pixels and their four entries in the center (highlighted in red) are aligned to those surrounding input conditions (i, j) in the timing table, where i and j are real numbers representing input transition time or load capacitance. The pixel of each map is associated with the timing parameter, row-index, or column-index at the corresponding position, and the pixels outside the table are assigned 0. For each map, the values of pixels are scaled between 0 and 1.

VAE model takes the table entries and their index values as input parameters and transforms them into a latent space, which in turn allows the input parameters to be reconstructed in the output layer. We use mean and deviation vectors of the latent space as input features.

Manuscript received 25 April 2023; revised 17 July 2023; accepted 25 August 2023. Date of publication 18 September 2023; date of current version 26 December 2023. This work was supported in part by the Institute of Information and Communications Technology Planning and Evaluation (IITP) Grant funded by the Korea Government (MSIT, Software Systems for AI Semiconductor Design) under Grant 2021-0-00754, and in part by the National Research Foundation of Korea (NRF) of MSIT under Grant 2019R1A2C2003402. This article was recommended by Associate Editor N. K. Jha. *(Corresponding author: Daijoon Hyun.)*

Daijoon Hyun is with the Department of Semiconductor Systems Engineering, Sejong University, Seoul 05006, South Korea (e-mail: djhyun@sejong.ac.kr).

Younggwang Jung and Youngsoo Shin are with the School of Electrical Engineering, KAIST, Daejeon 34141, South Korea (e-mail: ykj0503@kaist.ac.kr; youngsoo@kaist.edu).

Digital Object Identifier 10.1109/TCAD.2023.3316991

1937-4151 © 2023 IEEE. Personal use is permitted, but republication/redistribution requires IEEE permission.
See https://www.ieee.org/publications/rights/index.html for more information.

Authorized licensed use limited to: Thapar Institute of Engineering & Technology. Downloaded on March 23,2026 at 16:48:11 UTC from IEEE Xplore. Restrictions apply.

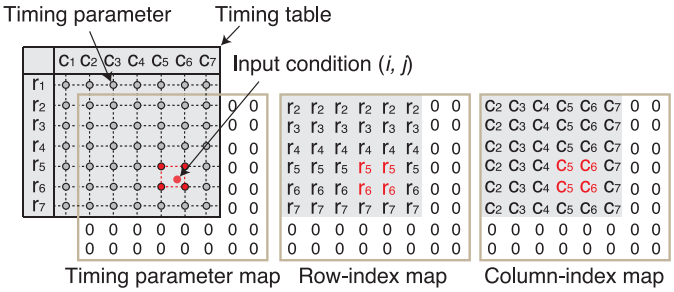

Fig. 1. Three input maps extracted from a timing table: timing parameter map, row-index map, and column-index map.

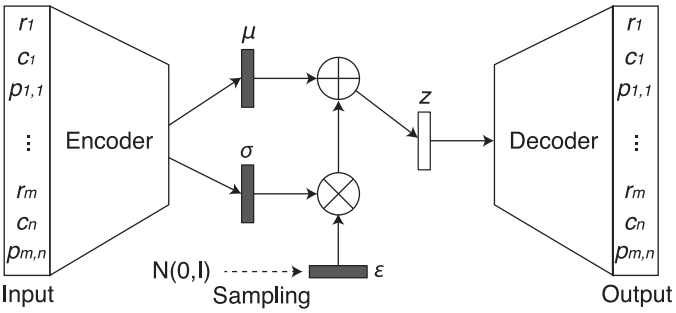

Fig. 2. Structure of VAE, which is used to extract the distribution vectors (μ and σ) of latent space from a timing table.

1) Variational Autoencoder: Fig. 2 illustrates a structure of VAE model. Assume that a table consists of m row-index, n column-index, and $m{\times}n$ timing parameters. For each timing parameter, its row-index and column-index are fed together into the input layer of the encoder, where each value is normalized by its greatest number. The encoder is an MLP, in which $3{\times}n{\times}m$ input parameters go through five hidden layers, gradually decreasing the number of neurons for each layer. It results in a mean vector ($\boldsymbol{\mu}$) and a deviation vector ($\boldsymbol{\sigma}$), both containing seven elements. A latent vector (z) is obtained through element-wise operation by

$$z = \boldsymbol{\mu} + \boldsymbol{\sigma} \cdot \boldsymbol{\epsilon} \tag{1}$$

where $\boldsymbol{\epsilon}$ is a random noise vector sampled from standard normal distributions $N(0, \boldsymbol{I})$. The latent vector is fed into a decoder with five hidden layers, in which the number of neurons is increased for each layer and $3{\times}n{\times}m$ outputs come out.

The VAE model is trained to minimize evidence lower bound (ELBO) [4]

$$\text{ELBO} = \sum_{i \in \boldsymbol{O}} (y_i - \hat{y}_i)^2 + \frac{1}{2} \sum_{j=1}^{|\boldsymbol{L}|} \left(\sigma_j^2 + \mu_j^2 - \ln\left(\sigma_j^2\right) - 1\right) \tag{2}$$

where $\boldsymbol{O}$ is a set of all outputs and $\boldsymbol{L}$ is a latent vector. y_i is the ith output, $\hat{y}_i$ is the corresponding input, and μ_j and σ_j are the jth elements of mean vector and deviation vector, respectively. The first term represents the error in how well the output reconstructs the input, called reconstruction loss, while the second term indicates the difference between the normal distributions of $\boldsymbol{\mu}$ and $\boldsymbol{\sigma}$ and $N(0, \boldsymbol{I})$, called regularization loss.

B. R-CNN Model

The proposed structure of R-CNN is illustrated in Fig. 3(a). It is composed of four recurrent convolutional layers (RCLs), where each RCL is made up of CNN and RNN. CNN takes the three input maps. As RCL increases, the edge pixels of the input maps are gradually replaced with zeros in such a way that the pixel values are only held around the center. Finally, the input maps of the fourth CNN contain only four nonzero pixel values around input condition (i, j).

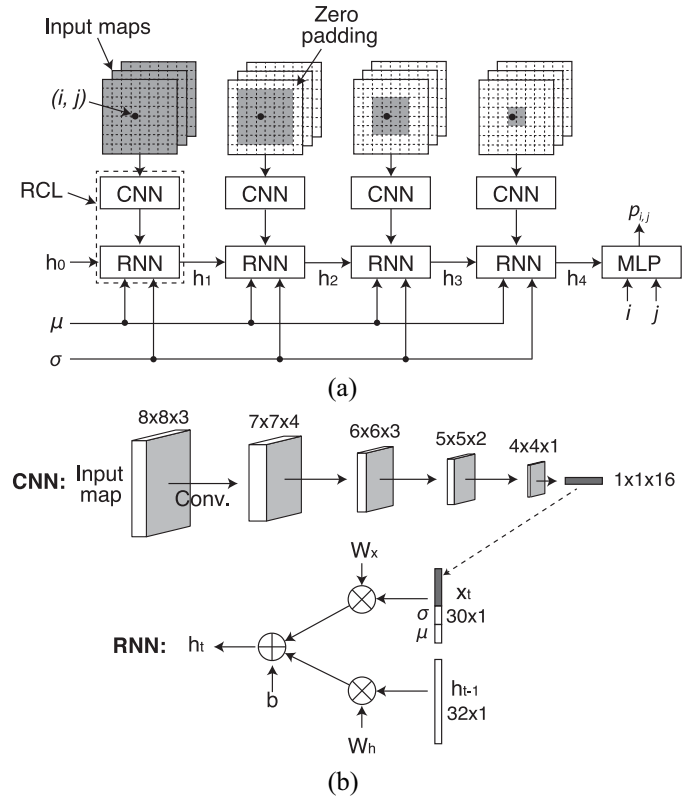

Fig. 3. (a) R-CNN model with four RCLs. (b) Structure of an RCL.

Fig. 3(b) shows the structure of an RCL, where the input maps go through four convolutional layers of CNN. By convolutions, the map size and the number of channels are gradually reduced, and 4×4 map is flattened to a vector of 16 bits at the output of CNN. It is concatenated to the distribution vectors, $\boldsymbol{\mu}$ and $\boldsymbol{\sigma}$, obtained by VAE, and this makes a feedforward input $\boldsymbol{x_t}$ of RNN. Recurrent input $\boldsymbol{h_{t-1}}$ comes from the previous layer, while $\boldsymbol{h_0}$ has all zeros. Recurrent output $\boldsymbol{h_t}$ is calculated by

$$\boldsymbol{h_t} = \tanh\left(\boldsymbol{W_h}\, \boldsymbol{h_{t-1}} + \boldsymbol{W_x}\, \boldsymbol{x_t} + \boldsymbol{b}\right) \tag{3}$$

where $\boldsymbol{W_h} \in \mathbb{R}^{32\times32}$ and $\boldsymbol{W_x} \in \mathbb{R}^{32\times30}$ are the weight matrices and $\boldsymbol{b}$ is a bias vector of 32 elements. The recurrent output $\boldsymbol{h_4}$ of the fourth RCL goes into MLP with two hidden layers and one output layer, where the number of neurons decreases to 16, 8, and 1, and LeakyReLu is used as the activation function for each layer. Sigmoid function and inverse scaling are applied to the output of MLP, and timing parameter $p_{i,j}$ is finally predicted.

During training, the table entries are used as training samples, and their mean-squared error (MSE) is used as a loss function

$$\text{Loss} = \sum_{\forall(i,j)} \left(p_{i,j} - \hat{p}_{i,j}\right)^2 \tag{4}$$

where (i, j) is a pair of input conditions, and $\hat{p}_{i,j}$ is the actual timing parameter at (i, j).

C. Runtime Reduction Methods

ML interpolation involves a large number of arithmetic operations, resulting in a longer processing time in comparison to bilinear interpolation. To minimize the increase in runtime, we execute the

Authorized licensed use limited to: Thapar Institute of Engineering & Technology. Downloaded on March 23,2026 at 16:48:11 UTC from IEEE Xplore. Restrictions apply.

TABLE I
ABSOLUTE ERROR DISTRIBUTIONS OF THE TIMING PARAMETERS OBTAINED BY BILINEAR INTERPOLATION AND ML INTERPOLATION

Corners	Bilinear interpolation									ML interpolation (Proposed)								
	Cell delay			Transition time			Constraint			Cell delay			Transition time			Constraint		
	μ (%)	3σ (%)	Max (%)	μ (%)	3σ (%)	Max (%)	μ (ps)	3σ (ps)	Max (ps)	μ (%)	3σ (%)	Max (%)	μ (%)	3σ (%)	Max (%)	μ (ps)	3σ (ps)	Max (ps)
c_1	0.1	1.8	3.4	0.3	4.1	14.8	3.0	10.8	13.8	0.0	0.2	0.7	0.1	0.5	1.3	2.1	7.3	8.9
c_2	1.3	7.8	10.9	1.5	9.8	13.2	3.7	13.3	18.1	0.1	0.7	2.5	0.1	0.7	1.8	2.2	7.4	10.5
c_3	1.3	11.7	18.5	2.0	12.3	16.1	2.7	9.5	12.4	1.2	3.9	4.2	0.6	1.7	2.4	2.1	7.4	9.2
c_4	1.7	7.4	18.8	2.1	10.0	14.1	2.3	8.1	10.0	1.1	4.3	4.5	0.5	1.6	1.9	2.0	7.5	9.0
c_5	1.7	14.6	22.1	2.0	11.0	13.3	2.1	7.5	10.1	1.0	4.7	5.1	0.5	1.6	1.8	1.9	7.5	9.3
c_6	2.1	13.8	28.2	2.1	10.3	13.0	2.6	9.2	11.6	1.4	4.0	4.0	0.5	1.7	1.8	2.1	7.6	8.7
c_7	0.5	5.8	52.5	0.4	2.3	4.4	2.8	10.0	13.5	0.1	1.3	6.2	0.1	0.6	1.3	2.1	7.5	9.8
c_8	0.7	7.3	27.4	0.6	4.3	12.0	2.5	8.8	11.2	0.2	1.1	1.3	0.2	1.0	1.9	2.1	7.2	9.2
c_9	0.7	7.3	32.8	0.5	3.0	9.4	2.2	7.9	10.0	0.2	1.7	8.7	0.2	0.9	1.2	1.9	7.5	9.3
Avg.	1.1	8.6	23.8	1.3	7.4	12.3	2.6	9.5	12.3	0.6	2.4	4.1	0.3	1.1	1.7	2.1	7.4	9.3

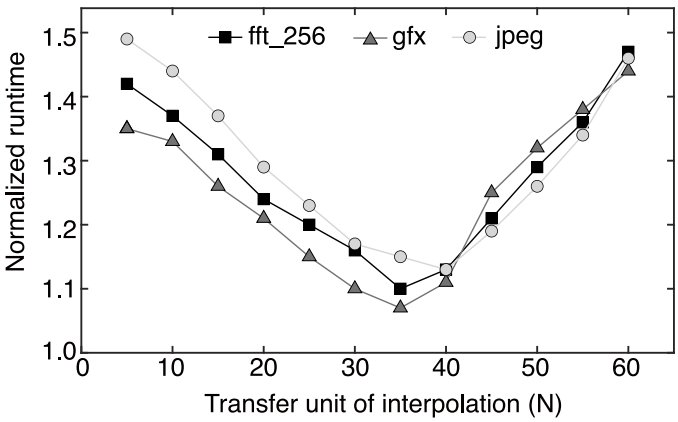

Fig. 4. Runtime of parallelized ML interpolation, normalized to that of bilinear interpolation, according to the transfer unit of interpolation (N).

arithmetic operations of VAE and RCLs beforehand. This enables ML interpolation to be performed by running only MLP.

Parallelizing ML interpolations in GPU also reduces the runtime overhead. In STA, logic cells in a given circuit are visited in topological order and ML interpolations are invoked to obtain their timing information. For every N request of interpolation, the required data is sent to the GPU at once so that N interpolations are performed in parallel. Once the GPU has completed the computations, the results are returned to the CPU, and the timing analysis from those cells is resumed. Parallel processing makes interpolation faster, but additional runtime is incurred due to the need to transfer data and the latency until N requests are collected. Fig. 4 illustrates the relationship between the transfer unit of interpolation (N) and the runtime of STA, which is normalized to that of bilinear interpolation. The figure shows that three test circuits have the lowest runtimes at similar values of N.

III. EXPERIMENTAL RESULTS

Experiments are conducted with 55-nm industrial NLDM library, where each corner library includes 1156 timing tables and the size of tables is 7-by-7. VAE is trained using the data of all tables in a timing library, where a noise vector is randomly sampled every epoch. R-CNN is generated for each table, in which the distribution vectors of the table, extracted by VAE, are used as input features. For both models, training is performed in units of 50 epochs and continues until the minimum loss is observed, called early stopping [5]. An Adam optimizer [6] is used for the training with a learning rate of 0.001. The models are implemented using PyTorch [7] package in Python, and its training and inference are performed in Nvidia TITAN Xp GPU.

In order to assess the effect of interpolation on circuit timing and runtime, sequential circuits from OpenCore [8] are synthesized with Design Compiler [9] and IC Compiler II [10]. The circuit timing is analyzed through OpenSTA [11], which is modified to apply various interpolation methods. In OpenSTA, the parallelization of ML interpolation is implemented with PyTorch API and performed on the same GPU hardware.

A. Assessment of Estimated Timing Parameters

1) Comparison With Bilinear Interpolation: Table I shows the absolute error distributions of timing parameters, estimated by bilinear interpolation and ML interpolation, where three parameters, mean (μ), 99.7th percentile (3σ), and the maximum error (Max), are used to represent the error distribution. For each corner library, 0.49M cell delays, 0.49M output transition times, and 34K constraints are estimated by two interpolation methods, and they are compared with simulation values. For cell delay and transition time, samples out of 3σ are filtered out if they have an error of less than 3 ps.

Bilinear interpolation uses linear interpolation in one direction first, and then again in another direction. This causes some errors for nonlinearly changing timing parameters. The error distributions of bilinear interpolation are represented in columns 2–10, where the cell delay error has μ of 1.1% and 3σ of 8.6% on average of test corners. μ and 3σ of output transition time are similar to those of cell delay, and constraint errors are as small as a few picoseconds. Large errors are observed in the input conditions of large input transition time and small load capacitance, where nonlinearity increases [12]; for example, 71% of the top 30 errors in c_3 corner occur under that condition. There is a large variation in the error distributions across test corners. Since timing characteristics are different for each corner, the maximum index values are set differently. If the ranges of row-index and column-index are wide, the interpolation error may increase due to the large interval between indices. Fig. 5 shows that our conjecture is correct, where the product of the maximum row-index and the maximum column-index is normalized to the smallest one and represented in the x-axis and the interpolation errors are shown in the y-axis.

ML interpolation using R-CNN achieves a large improvement in the accuracy of timing parameters. μ and 3σ of cell delay error are reduced to 0.6% and 2.4%, respectively, which are 0.5% and 6.2% smaller compared to bilinear interpolation. μ and 3σ of transition

Authorized licensed use limited to: Thapar Institute of Engineering & Technology. Downloaded on March 23,2026 at 16:48:11 UTC from IEEE Xplore. Restrictions apply.

TABLE II
COMPARISON OF BILINEAR INTERPOLATION AND ML INTERPOLATION IN TERMS OF PATH DELAY ERROR AND STA RUNTIME

Circuits	#Cells (k)	Bilinear interpolation							ML interpolation (Proposed)						
		Max delay error			Min delay error			Time (s)	Max delay error			Min delay error			ΔTime (%)
		μ (%)	3σ (%)	Max (%)	μ (ps)	3σ (ps)	Max (ps)		μ (%)	3σ (%)	Max (%)	μ (ps)	3σ (ps)	Max (ps)	
reed_sol	34	0.6	1.2	2.9	25	52	53	37	0.1	0.2	0.2	3	9	9	11
ethernet	54	0.6	1.3	1.8	26	182	188	56	0.1	0.2	0.2	4	17	18	23
vga_lcd	83	0.8	1.1	8.9	8	65	73	85	0.1	0.2	0.8	4	22	24	12
fft_128	280	0.7	1.1	2.2	23	314	341	121	0.1	0.2	0.2	9	64	68	12
tate_pair	319	0.5	1.1	7.0	25	66	69	72	0.1	0.2	0.2	5	14	15	17
fft_256	1146	0.7	2.4	2.7	43	159	163	764	0.1	0.1	0.1	5	19	22	7
gfx	1848	0.7	2.6	4.8	51	240	242	869	0.1	0.2	0.6	10	57	77	10
jpeg	5142	0.7	1.1	1.3	211	348	348	1175	0.1	0.2	0.3	71	80	81	10
gfx_mac	5449	0.8	1.2	3.3	55	185	435	1508	0.1	0.2	0.7	9	41	77	14
jpeg_mac	15384	0.8	1.2	3.0	133	362	399	2001	0.1	0.2	0.7	76	87	88	13
Average		0.7	1.4	3.8	60	197	231		0.1	0.2	0.4	20	41	48	13

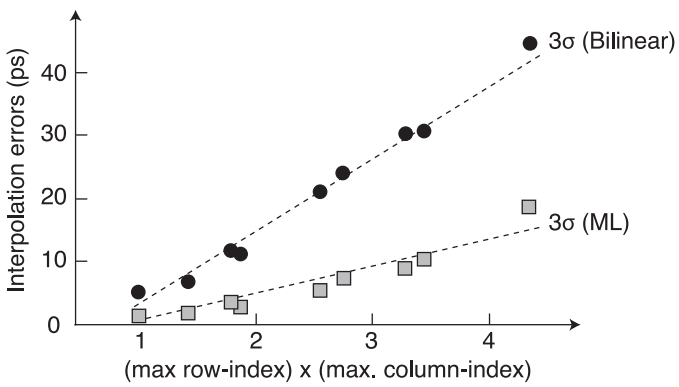

Fig. 5. Correlation between the interpolation error (y-axis) and the normalized product of the maximum row-index and the maximum column-index (x-axis).

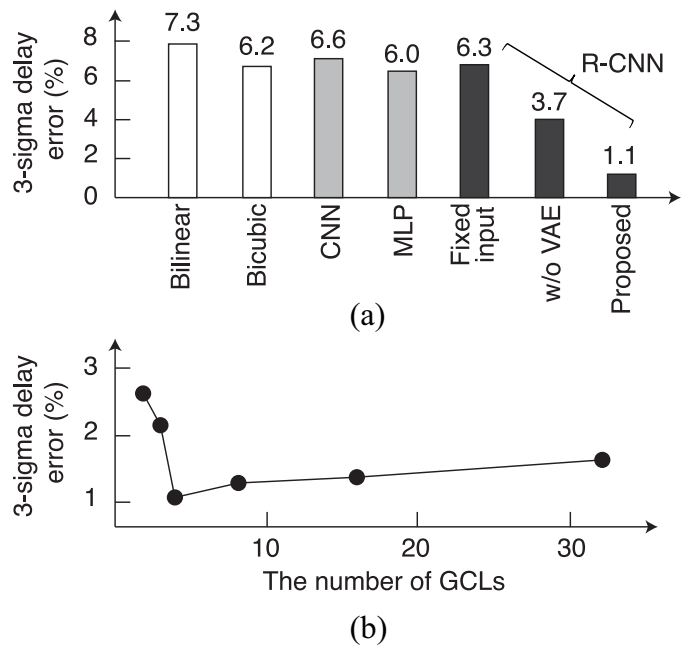

Fig. 6. Comparison of cell delay errors (a) for various interpolation methods and R-CNN variants and (b) for different numbers of GCLs.

time error are reduced by 1.0% and 6.3%, respectively. The distribution parameters of constraint are also improved by a few picoseconds. This improvement in accuracy is achieved by using a well-trained nonlinear model with full parameters from the timing table. 3σ of cell delay ranges from 0.2% to 4.7% for all test corners. This is also explained by the index range of the timing table, as shown in Fig. 5 (see gray squares).

2) Comparison to Various Interpolation Methods: Fig. 6(a) shows 3σ cell delay errors at c_8 corner for various interpolation methods, where the interpolation errors using mathematical functions are represented in white color and those using ML models are shown as gray bars. Bicubic interpolation uses a cubic polynomial function to fit consecutive table entries. This is performed in x- and y-direction to find an unknown value. It is typically known to be more accurate than bilinear interpolation, but the improvement is only 1.1%. Even though all table entries are considered with nonlinear model, the accuracy of CNN and MLP is not improved a lot; this may be due to the lack of training samples compared to the number of weights. The errors of R-CNN variants are represented as black bars. The proposed method achieves 3σ delay error of 1.1%, and the error increases to 6.3% if the same input maps are fed into all RCLs without zero padding (Fixed input). The distribution vectors from VAE affect the accuracy by 2.6% (see w/o VAE).

The number of RCLs also has an impact on accuracy as shown in Fig. 6(b), which shows the 3σ delay error for the number of layers. A small number of layers (2 or 3 layers) does not provide enough information about the timing parameter distributions. In addition, a recurrent network has the disadvantage of degrading long-term memory as the number of layers increases. As a result, the best accuracy is observed for R-CNN with four layers.

B. Impact on Circuit Timing

For a timing path, the delay errors of the cells on it are accumulated, and the transition time error further increases the delay errors through the interpolation under bad input conditions. Table II compares bilinear interpolation and ML interpolation in terms of the absolute error distribution of path delays and STA runtime (Time). We measure the maximum and minimum path delays from $5k$ setup critical paths and $1k$ hold critical paths for each circuit, respectively, and they are compared with those obtained by bilinear interpolation with a 50-by-50 timing table, which serves as a reference.

The absolute percentage error distribution of maximum delay using bilinear interpolation is represented in columns 3–5 of Table II. Max of 8.9% is observed in vga_lcd circuit, where the path delay of bilinear interpolation is 930 ps smaller compared to the reference; this can cause an unexpected setup time violation even after timing closure.

Authorized licensed use limited to: Thapar Institute of Engineering & Technology. Downloaded on March 23,2026 at 16:48:11 UTC from IEEE Xplore. Restrictions apply.

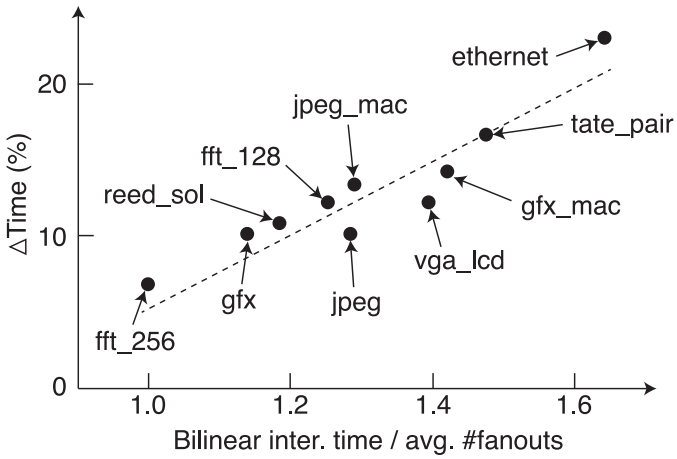

Fig. 7. Runtime increase (ΔTime) for STA according to the bilinear interpolation runtime divided by the average number of fanouts.

On the other hand, the maximum delay error is significantly reduced by ML interpolation as shown in columns 10–12; specifically, 3σ is 0.2% on average of test circuits, and the worst value of Max is less than 1%. The error distribution of minimum delay using bilinear interpolation is represented with a unit of picosecond in columns 6–8. Even though hold critical path consists of a small number of cells, the error is not small; for example, 435-ps error in gfx_mac is caused by eight cells. Hold slack is calculated more pessimistically because bilinear interpolation makes the path delay smaller. This can increase the number of hold buffers, increasing power consumption and circuit area.

C. Analysis of Runtime

The runtime of STA using bilinear interpolation is shown in column 9 of Table II. On average of test circuits, bilinear interpolation accounts for 4.4% of total STA runtime. Since the number of arithmetic operations in MLP of R-CNN is roughly 50 times greater than bilinear interpolation, the runtime increases significantly; for instance, fft_256 takes 2.8 times runtime (2129 s) with ML interpolation. To reduce the runtime overhead, ML interpolation is parallelized in GPU, and the runtime increase (ΔTime) is reduced to 13% on average of test circuits, as shown in the last column. For reference, parallelizing ML interpolation in 16-thread of CPU gives 26% runtime increase. Bilinear interpolation has a small number of operations and thus does not benefit from parallelization. Rather, the runtime increases due to the data transfer time. For example, parallel bilinear interpolation in GPU takes 926 s in fft_256, which is 21% longer than before.

For test circuits, ΔTime ranges from 7% (fft_256) to 23% (Ethernet). Fig. 7 shows that ΔTime is proportional to the rate of bilinear interpolation in total STA runtime and inversely proportional to the average number of fanouts. The runtime increase due to ML interpolation occurs as much as the number of interpolations requested, and the greater the number of fanouts in a circuit, the more interpolations can be processed simultaneously, thus benefiting from parallelism.

IV. Conclusion

We have addressed ML interpolation to get more accurate timing parameters. The R-CNN model is employed to allow that the interpolation can be affected by various ranges of table entries, and VAE is used to extract the distribution features of the table. Additionally, ML interpolations are parallelized in GPU to decrease the runtime overhead of many operations. Experimental results demonstrate that ML interpolation reduces timing parameter error by 19.7% and path delay error by 3.4% in comparison to bilinear interpolation at the cost of 13% runtime overhead on average of test circuits.

ML interpolation can be utilized efficiently in conjunction with bilinear interpolation in such a way that ML interpolation is only applied in situations where there are large nonlinearities. It can also be applied to other applications, such as the timing library of the current source model and the power library including the information of internal energy. They are included in our future work.

Acknowledgment

The EDA Tool was supported by the IC Design Education Center.

References

[1] R. Chadha and J. Bhasker, *Static Timing Analysis for Nanometer Designs: A Practical Approach*, 1st ed., New York, NY, USA: Springer, 2009.
[2] W. Raslan and Y. Ismail, "Deep-learning cell-delay modeling for static timing analysis," *Ain Shams Eng. J.*, vol. 14, no. 1, Feb. 2023, Art. no. 101828.
[3] D. Garyfallou, A. Vagenas, C. Antoniadis, Y. Massoud, and G. Stamoulis, "Leveraging machine learning for gate-level timing estimation using current source models and effective capacitance," in *Proc. Great Lakes Symp. VLSI*, Jun. 2022, pp. 77–83.
[4] D. P. Kingma and M. Welling, "Auto-encoding variational Bayes," Dec. 2022, *arXiv:1312.6114*.
[5] Y. Yao, L. Rosasco, and A. Caponnetto, "On early stopping in gradient descent learning," *Constr. Approx.*, vol. 26, pp. 289–315, Apr. 2007.
[6] D. P. Kingma and J. Ba, "Adam: A method for stochastic optimization," Dec. 2017, *arXiv:1412.6980*.
[7] A. Paszke et al., "Pytorch: An imperative style, high-performance deep learning library," *Adv. Neural Inf. Process. Syst.*, vol. 32, pp. 8026–8037, Dec. 2019.
[8] "OpenCores." Accessed: May 21, 2021. [Online]. Available: http://www.opencores.org
[9] *Design Compiler User Guide*, Synopsys, Sunnyvale, CA, USA, Jun. 2015.
[10] *IC Compiler II User Guide*, Synopsys, Sunnyvale, CA, USA, Dec. 2021.
[11] "OpenSTA." Accessed: Jun. 30, 2022. [Online]. Available: https://github.com/The-OpenROAD-Project/OpenSTA
[12] S. Miryala, B. Kaur, B. Anand, and S. Manhas, "Efficient nanoscale VLSI standard cell library characterization using a novel delay model," in *Proc. Int. Symp. Qual. Electron. Design*, Mar. 2011, pp. 1–6.

Authorized licensed use limited to: Thapar Institute of Engineering & Technology. Downloaded on March 23,2026 at 16:48:11 UTC from IEEE Xplore. Restrictions apply.